

# Comunicazione tra Identity Broker e servizi SSO

---

**Destinatari:** team esterni che gestiscono un servizio SSO (Authorization Server OAuth2 / OpenID Provider) con cui il **Prologic Identity Broker** deve dialogare, oppure che devono comprendere il canale di comunicazione per attività di integrazione, sicurezza o rete.

**Perimetro:** questo documento descrive **solo** il dialogo tra Identity Broker e il servizio SSO (provider di identità).

**Provider trattati:** **Silfi IAM (WSO2)** e **Microsoft Entra ID**. Il Broker supporta anche altri provider, non coperti qui.

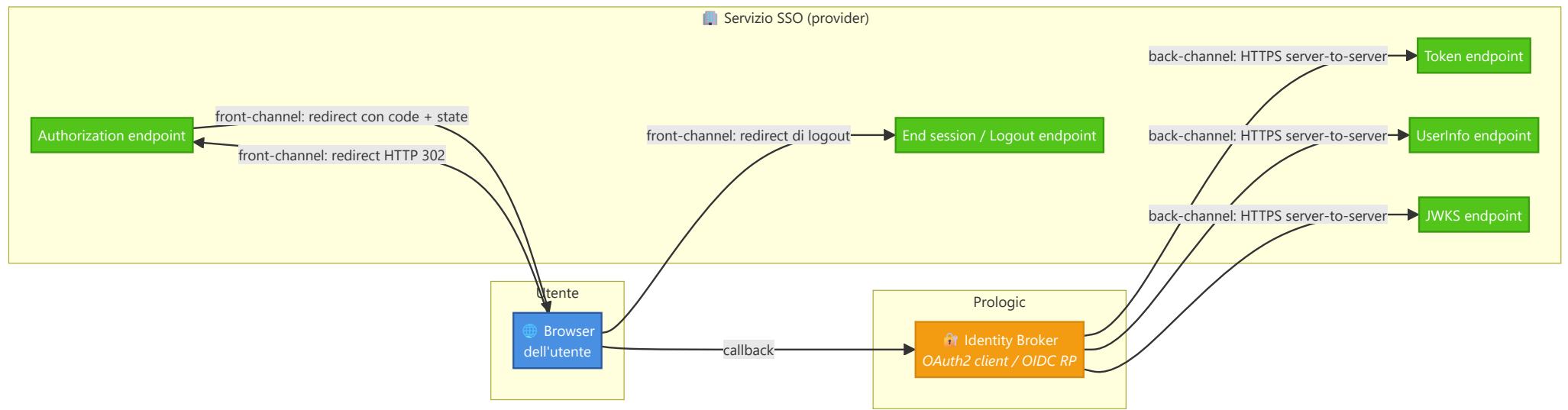
---

## 1. In breve

L'Identity Broker si comporta, verso il servizio SSO, come un **client OAuth2 confidenziale** e **Relying Party OpenID Connect** che implementa l'**Authorization Code Flow**:

- **È il Broker**, e non l'applicazione finale, il client registrato presso il servizio SSO. Le applicazioni client non parlano mai direttamente con il provider e non conoscono `client_id` / `client_secret`.
- **Esiste una sola** `redirect_uri` registrata presso il provider: l'endpoint di callback del Broker. Non cambia in funzione dell'applicazione finale né dell'ente.
- Il Broker si autentica al *token endpoint* con `client_id` + `client_secret` **nel body** della richiesta (`client_secret_post`), sia con Silfi sia con Entra ID.
- Dopo l'autenticazione, il Broker **normalizza** gli attributi utente ricevuti dal provider in un modello comune ed emette un proprio token; l'`access_token` del provider **non** viene propagato alle applicazioni finali.
- La configurazione del provider è **per ente** (multi-tenant): lo stesso Broker può dialogare con più servizi SSO, o con lo stesso servizio SSO ma con client e parametri differenti per ciascun ente.

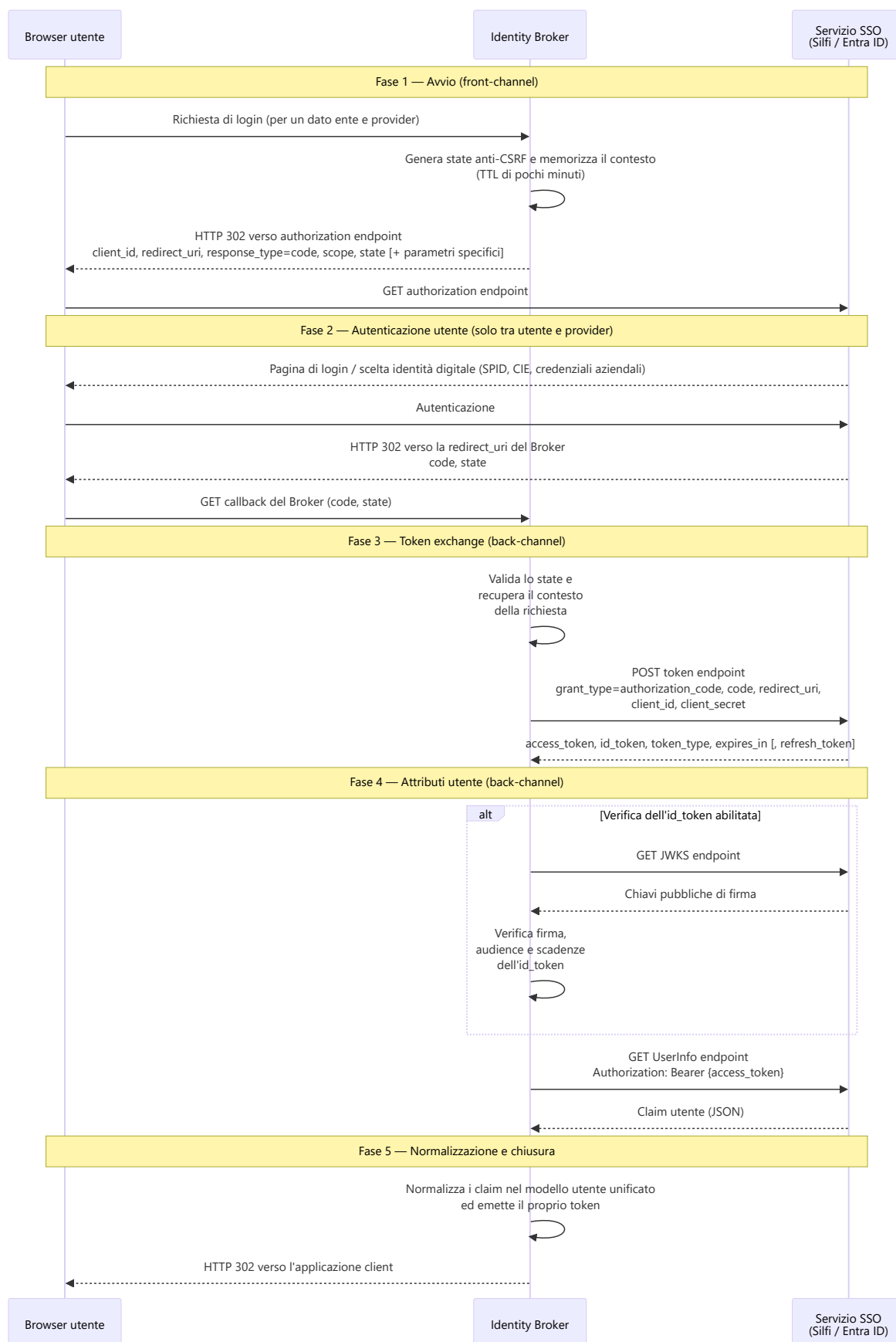
2. Attori e canali



Due canali distinti, con implicazioni diverse per rete e firewall:

| Canale        | Chi effettua la chiamata                             | Verso                                            | Note                                                                                                      |
|---------------|------------------------------------------------------|--------------------------------------------------|-----------------------------------------------------------------------------------------------------------|
| Front-channel | Il <b>browser</b> dell'utente, via redirect HTTP 302 | Authorization endpoint, Logout endpoint          | Richiede che l'utente raggiunga il provider; nessun traffico diretto dai server del Broker                |
| Back-channel  | Il <b>server</b> dell'Identity Broker                | Token endpoint, UserInfo endpoint, JWKS endpoint | Richiede <b>egress HTTPS</b> dai nodi del Broker verso il provider; nessun credenziale utente in transito |

### 3. Schema del flusso di autenticazione



Il dialogo con il provider **si chiude alla fase 5**: tutto ciò che avviene dopo (consegna del token all'applicazione client, validazione, sessioni) resta interno al perimetro Broker → applicazione.

## 4. Dettaglio delle chiamate verso il provider

| # | Endpoint del provider             | Chi chiama                        | Metodo                                                 | Parametri inviati                                                                                                                                                   | Autenticazione del client                              |
|---|-----------------------------------|-----------------------------------|--------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------------------------|
| 1 | <b>Authorization</b>              | Browser (via redirect del Broker) | GET                                                    | <code>client_id</code> , <code>redirect_uri</code> , <code>response_type=code</code> , <code>scope</code> , <code>state</code> , + parametri specifici del provider | Nessuna (client pubblico sul front-channel)            |
| 2 | <b>Token</b> (authorization code) | Broker (server-to-server)         | POST<br><code>application/x-www-form-urlencoded</code> | <code>grant_type=authorization_code</code> , <code>code</code> , <code>redirect_uri</code> , <code>client_id</code> , <code>client_secret</code>                    | <code>client_secret_post</code> (credenziali nel body) |
| 3 | <b>JWKS</b>                       | Broker (server-to-server)         | GET                                                    | —                                                                                                                                                                   | Nessuna (endpoint pubblico)                            |
| 4 | <b>UserInfo</b>                   | Broker (server-to-server)         | GET                                                    | Header <code>Authorization: Bearer {access_token}</code> , <code>Accept: application/json</code>                                                                    | Access token dell'utente                               |
| 5 | <b>Token</b> (refresh)            | Broker (server-to-server)         | POST<br><code>application/x-www-form-urlencoded</code> | <code>grant_type=refresh_token</code> , <code>refresh_token</code> , <code>client_id</code> , <code>client_secret</code>                                            | <code>client_secret_post</code> (credenziali nel body) |
| 6 | <b>Logout / End session</b>       | Browser (via redirect)            | GET                                                    | <code>id_token_hint</code> , <code>post_logout_redirect_uri</code>                                                                                                  | Nessuna                                                |

### Note sul comportamento del Broker

- **redirect\_uri** **unica e coerente**. Il Broker invia sempre lo **stesso** valore di `redirect_uri` (il proprio endpoint di callback, senza parametri aggiuntivi in query) sia nella chiamata di authorization sia nel token exchange, perché i provider richiedono corrispondenza esatta tra le due. La destinazione finale dell'utente è gestita dal Broker e non compare mai nella `redirect_uri` registrata presso il provider.
- **state**. Sempre presente, generato dal Broker, con contesto della richiesta conservato server-side e TTL nell'ordine dei minuti. Il provider deve restituirlo **invariato** sul callback: un `state` assente, alterato o scaduto interrompe il flusso.
- **response\_type**. Solo `code` (Authorization Code Flow). Non vengono usati flussi impliciti o ibridi.
- **PKCE**. Attualmente **non** utilizzato: il Broker è un client confidenziale con segreto server-side. Se il provider impone PKCE obbligatorio per il client registrato, l'integrazione va concordata preventivamente.
- **scope**. Obbligatorio e definito nella configurazione del provider per ciascun ente; il Broker non applica valori di default impliciti.
- **Refresh token**. Usato solo se il provider lo supporta e se il Broker lo riceve nella risposta del token endpoint. Ogni refresh è seguito da una nuova lettura degli attributi utente, per mantenere il profilo allineato.
- **Logout**. Il Broker costruisce l'URL di logout del provider con `id_token_hint` e `post_logout_redirect_uri` e lo restituisce all'applicazione, che effettua il redirect del browser. Il logout sul provider è quindi **front-channel**; il Broker non effettua chiamate back-channel di revoca verso il provider. Se il provider non espone un endpoint di logout configurato, il logout resta locale al Broker.

## 5. Verifica dell' `id_token` (OIDC)

Quando nella configurazione dell'ente è presente l'URI del JWKS del provider, il Broker **verifica** l' `id_token` prima di usarne i claim:

- **Algoritmo**: firma **RSA (RS256)** con chiave pubblica recuperata dal JWKS.

- **Header `kid` obbligatorio:** l' `id_token` deve indicare l'identificativo della chiave di firma; senza `kid` la verifica non è possibile.
- **Controlli effettuati:** firma, **audience** (deve contenere il `client_id` del Broker), validità temporale (`exp` / `nbf`).
- **Chiavi in cache:** il JWKS viene recuperato e messo in cache; una rotazione delle chiavi va resa disponibile sul JWKS endpoint con il consueto periodo di sovrapposizione, altrimenti si generano fallimenti di verifica transitori.

**Attenzione:** se la verifica non va a buon fine, il Broker **non interrompe il flusso** ma ricade sulla sola lettura dei claim dell' `id_token` senza validazione della firma. È quindi importante che il JWKS endpoint sia raggiungibile e che le chiavi siano allineate, altrimenti si perde silenziosamente un controllo di sicurezza.

## Da dove provengono i claim utente

Il Broker può leggere gli attributi dell'utente da tre sorgenti, selezionabili in configurazione per provider:

| Sorgente                           | Comportamento                                                                                                | Quando usarla                                                                                 |
|------------------------------------|--------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------|
| <b>UserInfo endpoint</b> (default) | Chiamata <code>GET</code> al UserInfo endpoint con l'access token                                            | Provider che espongono tutti gli attributi necessari via UserInfo (caso tipico di Silfi)      |
| <code>id_token</code>              | I claim vengono letti dall' <code>id_token</code> restituito dal token endpoint; nessuna chiamata a UserInfo | Provider che non espongono, o espongono in modo parziale, UserInfo; richiede JWKS configurato |
| <b>Merge</b>                       | Claim dall' <code>id_token</code> , poi UserInfo che prevale sui campi in conflitto                          | Attributi distribuiti tra <code>id_token</code> e UserInfo; richiede JWKS configurato         |

## 6. Attributi attesi e normalizzazione

Il Broker mappa i claim del provider su un modello utente unificato:

| Campo unificato             | Significato                                         | Note                                                                             |
|-----------------------------|-----------------------------------------------------|----------------------------------------------------------------------------------|
| <code>providerUserId</code> | Identificativo dell'utente presso il provider       | Tipicamente dal claim <code>sub</code>                                           |
| <code>email</code>          | Indirizzo email                                     | Può essere assente a seconda del provider e della modalità di autenticazione     |
| <code>givenName</code>      | Nome                                                | Da <code>given_name</code> ; se assente si usa <code>name</code> (nome completo) |
| <code>familyName</code>     | Cognome                                             | Da <code>family_name</code>                                                      |
| <code>fiscalCode</code>     | Codice fiscale                                      | Dipende dal provider (vedi §7)                                                   |
| <code>ente</code>           | Ente di appartenenza del contesto di autenticazione | Determinato dal Broker, non dal provider                                         |

Il Broker genera inoltre un **identificativo utente stabile** derivato da provider + identificativo stabile dell'utente + ente. Di conseguenza:

**Requisito importante:** l'attributo usato come identificativo stabile **non deve cambiare** tra autenticazioni successive dello stesso utente. Se il `sub` del provider non è stabile, va reso disponibile un attributo alternativo stabile (es. codice fiscale) e concordato il suo utilizzo.

## 7. Specificità per provider

### 7.1 Silfi IAM (WSO2)

Sistema federato usato dagli enti per l'autenticazione dei cittadini (SPID, CIE, altre identità digitali).

| Aspetto                              | Comportamento                                                                                                                                                                                               |
|--------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Endpoint                             | <code>authorize</code> , <code>token</code> , <code>userinfo</code> , <code>logout</code> (OIDC end session), JWKS opzionale                                                                                |
| Autenticazione al token endpoint     | <code>client_id</code> e <code>client_secret</code> <b>nel body</b> (Silfi non usa HTTP Basic)                                                                                                              |
| Parametri custom sull'authorization  | <code>agEntityId</code> ( <b>obbligatorio</b> , entity id dell'ente) e <code>comEntityId</code> (opzionale, entity id del comune), aggiunti ai parametri OAuth2 standard                                    |
| Sorgente claim                       | UserInfo endpoint                                                                                                                                                                                           |
| Claim utilizzati (profilo CITTADINO) | <code>sub</code> (identificativo OAuth2), <code>cn</code> ( <b>codice fiscale</b> ), <code>given_name</code> , <code>family_name</code> , <code>email</code> (presente <b>solo</b> con autenticazione SPID) |
| Identificativo stabile               | Il <code>sub</code> restituito da Silfi <b>non è stabile</b> tra autenticazioni diverse dello stesso utente: il Broker usa il <b>codice fiscale (cn)</b> per derivare l'identificativo utente stabile       |
| Refresh token                        | <b>Non supportato</b> : alla scadenza serve una nuova autenticazione completa                                                                                                                               |
| Logout                               | Redirect all'endpoint di logout con <code>id_token_hint</code> e <code>post_logout_redirect_uri</code>                                                                                                      |

## 7.2 Microsoft Entra ID

| Aspetto                                 | Comportamento                                                                                                                                                                                                        |
|-----------------------------------------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Endpoint                                | Endpoint Microsoft identity platform v2.0, <b>tenant-specifici</b> : il <code>tenantId</code> è parte del path delle URL ( <code>authorize</code> , <code>token</code> , JWKS, <code>logout</code> )                 |
| Autenticazione al token endpoint        | <code>client_id</code> e <code>client_secret</code> <b>nel body</b>                                                                                                                                                  |
| Parametri aggiuntivi sull'authorization | <code>prompt=select_account</code> (forza la scelta dell'account, evitando il single sign-on silenzioso dell'ultimo utente)                                                                                          |
| Sorgente claim                          | Configurabile: UserInfo endpoint (default), <code>id_token</code> , oppure merge dei due                                                                                                                             |
| Email                                   | Risolta in ordine: claim configurato esplicitamente, poi <code>upn</code> , <code>preferred_username</code> , <code>email</code> , <code>mail</code>                                                                 |
| Nome                                    | <code>given_name</code> / <code>family_name</code> ; se <code>given_name</code> è assente si usa <code>name</code>                                                                                                   |
| Codice fiscale                          | <b>Non previsto</b> nei claim standard Entra ID. Resta vuoto a meno che il tenant esponga un claim dedicato (anche tramite claim mapping / extension attribute) e il nome di quel claim venga configurato nel Broker |
| Refresh token                           | <b>Supportato</b> ( <code>grant_type=refresh_token</code> )                                                                                                                                                          |
| Logout                                  | Redirect all'endpoint di end session con <code>id_token_hint</code> e <code>post_logout_redirect_uri</code>                                                                                                          |
| JWKS                                    | Endpoint di discovery delle chiavi del tenant; raccomandato configurarlo, <b>obbligatorio</b> se i claim vengono letti dall' <code>id_token</code>                                                                   |